UPnP 포트 포워딩 분석

개요

rqbit의 UPnP 포트 포워딩 기능은 `librqbit-upnp` 크레이트에서 구현되며, 자동으로 라우터에서 포트 포워딩을 설정하여 외부에서 BitTorrent 클라이언트에 접근할 수 있도록 합니다. 이 기능은 NAT 환경에서 인바운드 연결을 가능하게 하여 피어 연결성을 크게 향상시킵니다.

핵심 구성 요소

1. UPnP 디바이스 검색 (SSDP)

SSDP 검색 메커니즘

```
const SSDP_MULTICAST_IP: SocketAddr =

SocketAddr::V4(SocketAddrV4::new(Ipv4Addr::new(239, 255, 255, 250), 1900));

pub const SSDP_SEARCH_WAN_IPCONNECTION_ST: &str = "urn:schemas-upnp-org:service:
```

```
pub fn make_ssdp_search_request(kind: &str) -> String {

format!(

"M-SEARCH * HTTP/1.1\r\n\

Host: 239.255.255.250:1900\r\n\

Man: \"ssdp:discover\"\r\n\

MX: 3\r\n\
```

```
ST: {kind}\r\n\

\r\n"

)

}
```

주요 특징:

- **멀티캐스트 검색**: 239.255.255.250:1900으로 SSDP 검색 요청 전송
- **서비스 타입 필터링**: WANIPConnection 서비스만 검색하여 효율성 향상
- **HTTP 기반 프로토콜**: 표준 HTTP 헤더를 사용한 검색 요청

검색 응답 파싱

```
pub fn parse_upnp_discover_response(

buf: &[u8],

received_from: SocketAddr,

) -> anyhow::Result<UpnpDiscoverResponse> {

let mut headers = [httparse::EMPTY_HEADER; 16];

let mut resp = httparse::Response::new(&mut headers);

resp.parse(buf).context("error parsing response")?;


// Location 헤더에서 디바이스 설명 URL 추출

let mut location = None;

for header in resp.headers {

match header.name {
```

```
"location" | "LOCATION" | "Location" => {
location = Some(std::str::from_utf8(header.value)?);
}
_ => continue,
}
}
let location = Url::parse(location.context("missing location header")?)?;
Ok(UpnpDiscoverResponse { location, received_from })
}
```

검색 과정:

1. UDP 멀티캐스트로 M-SEARCH 요청 전송
2. 라우터/게이트웨이에서 HTTP 응답으로 디바이스 정보 반환
3. Location 헤더에서 디바이스 설명 XML URL 추출
4. 응답을 받은 소켓 주소 기록 (로컬 IP 결정용)

2. 디바이스 설명 파싱

XML 구조 분석

```
#[derive(Clone, Debug, Deserialize, PartialEq, Eq)]
pub struct RootDesc {
#[serde(rename = "device")]
pub devices: Vec<Device>,
}
```

```
#[derive(Debug, Clone, Deserialize, PartialEq, Eq)]

pub struct Device {

#[serde(rename = "deviceType")]

pub device_type: String,

#[serde(rename = "friendlyName", default)]

pub friendly_name: String,

#[serde(rename = "serviceList", default)]

pub service_list: ServiceList,

#[serde(rename = "deviceList", default)]

pub device_list: DeviceList,

}
```

계층적 디바이스 구조:

- **InternetGatewayDevice**: 최상위 게이트웨이 디바이스
- **WANDevice**: WAN 연결 관리 디바이스
- **WANConnectionDevice**: 실제 연결 관리 디바이스
- **WANIPConnection 서비스**: 포트 포워딩 제어 서비스

서비스 검색 및 반복

```
impl Device {

pub fn iter_services(&self, parent: Span) -> Box<dyn Iterator<Item = (tracing::S

let self_span = self.span(parent);

let services = self.service_list.services.iter().map({
```

```
let self_span = self_span.clone();
move |s| (s.span(self_span.clone()), s)
});
// 재귀적으로 하위 디바이스의 서비스도 검색
Box::new(services.chain(self.device_list.devices.iter().flat_map({
let self_span = self_span.clone();
move |d| d.iter_services(self_span.clone())
})))
}
}
```

3. 로컬 IP 주소 결정

네트워크 인터페이스 분석

```
pub fn get_local_ip_relative_to(
local_dest: SocketAddr,
interfaces: &[NetworkInterface],
) -> anyhow::Result<IpAddr> {
fn masked_v4(ip: Ipv4Addr, mask: Ipv4Addr) -> u32 {
ip.to_bits() & mask.to_bits()
}
```

```
for i in interfaces {
for addr in i.addr.iter() {
match (local_dest, addr.ip(), addr.netmask()) {
// 자기 자신에게 연결하는 경우
(l, a, _) if l.ip() == a => return Ok(addr.ip()),
// IPv4 서브넷 마스크 매칭
(SocketAddr::V4(l), IpAddr::V4(a), Some(IpAddr::V4(m)))
if masked_v4(*l.ip(), m) == masked_v4(a, m) => {
return Ok(addr.ip());
}
// IPv6 링크 로컬 주소 처리
(SocketAddr::V6(l), IpAddr::V6(a), _)
if l.ip().is_unicast_link_local() && l.scope_id() > 0 => {
if a.is_unicast_link_local() && l.scope_id() == i.index {
return Ok(addr.ip());
}
}
_ => continue,
}
}
}
bail!("couldn't find a local ip address")
```

```
}
```

IP 주소 결정 로직:

1. **직접 매칭**: 목적지와 동일한 IP 주소
2. **서브넷 매칭**: 같은 서브넷 내의 IP 주소
3. **링크 로컬**: IPv6 링크 로컬 주소 처리
4. **폴백**: IPv6의 경우 첫 번째 발견된 주소 사용

4. 포트 매핑 요청

SOAP 요청 생성

```
async fn forward_port(
control_url: Url,
local_ip: IpAddr,
port: u16,
lease_duration: Duration,
) -> anyhow::Result<()> {
let request_body = format!(
r#"
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"
s:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<s:Body>
<u:AddPortMapping xmlns:u="{SERVICE_TYPE_WAN_IP_CONNECTION}">
<NewRemoteHost></NewRemoteHost>
```

```
<NewExternalPort>{port}</NewExternalPort>

<NewProtocol>TCP</NewProtocol>

<NewInternalPort>{port}</NewInternalPort>

<NewInternalClient>{local_ip}</NewInternalClient>

<NewEnabled>1</NewEnabled>

<NewPortMappingDescription>rust UPnP</NewPortMappingDescription>

<NewLeaseDuration>{}</NewLeaseDuration>

</u:AddPortMapping>

</s:Body>

</s:Envelope>

"#,

lease_duration.as_secs()

);



let client = reqwest::Client::new();

let response = client

.post(control_url.clone())

.header("Content-Type", "text/xml")

.header("SOAPAction", format!("\"{}#AddPortMapping\"", SERVICE_TYPE_WAN_IP_CONNE

.body(request_body)

.send()

.await?;
```

```
}
```

SOAP 요청 구조:

- **AddPortMapping**: UPnP 표준 포트 매핑 액션
- **외부/내부 포트**: 동일한 포트 번호 사용
- **프로토콜**: TCP 프로토콜 지정
- **리스 기간**: 포트 매핑 유효 기간 설정
- **설명**: "rust UPnP"로 매핑 식별

포트 포워더 관리자

1. UpnpPortForwarder 구조체

```
pub struct UpnpPortForwarder {
ports: Vec<u16>,
opts: UpnpPortForwarderOptions,
bind_device: Option<BindDevice>,
}
```

```
pub struct UpnpPortForwarderOptions {
pub lease_duration: Duration, // 기본: 60초
pub discover_interval: Duration, // 기본: 60초
pub discover_timeout: Duration, // 기본: 10초
}
```

2. 비동기 관리 루프

```
pub async fn run_forever(self) -> ! {
let (discover_tx, mut discover_rx) = unbounded_channel();
let discovery = self.discovery(discover_tx);

let mut spawned_tasks = HashSet::<Url>::new();
let mut endpoints = FuturesUnordered::new();
let mut service_managers = FuturesUnordered::new();

loop {
tokio::select! {
// 주기적 디바이스 검색
_ = &mut discovery => {},
// 새로운 디바이스 발견 처리
r = discover_rx.recv() => {
let r = r.unwrap();
endpoints.push(self.parse_endpoint(r));
},
// 엔드포인트 파싱 완료 처리
Some(Ok(endpoint)) = endpoints.next() => {
for (span, control_url) in endpoint.get_wan_ip_control_urls() {
if !spawned_tasks.contains(&control_url) {
```

```
let local_ip = endpoint.my_local_ip()?;

spawned_tasks.insert(control_url.clone());

service_managers.push(

self.manage_service(control_url, local_ip).instrument(span)

);

}

}

},

// 서비스 관리자 완료 처리

_ = service_managers.next() => {},

}

}

}
```

3. 포트 관리 및 갱신

```
async fn manage_port(&self, control_url: Url, local_ip: IpAddr, port: u16) -> !

let lease_duration = self.opts.lease_duration;

let mut interval = tokio::time::interval(lease_duration / 2);

interval.set_missed_tick_behavior(tokio::time::MissedTickBehavior::Delay);



loop {

interval.tick().await;
```

```
if let Err(e) = forward_port(control_url.clone(), local_ip, port, lease_duration
warn!("failed to forward port: {e:#}");
}
}
}
```

포트 관리 특징:

- **주기적 갱신**: 리스 기간의 절반마다 갱신 요청
- **오류 복구**: 실패 시 경고 로그 후 계속 시도
- **무한 루프**: 포트 매핑을 지속적으로 유지

오류 처리 및 복구 전략

1. 검색 단계 오류 처리

```
async fn discover_once(&self, tx: &UnboundedSender<UpnpDiscoverResponse>) -> any
// UDP 소켓 바인딩 실패 처리
let socket = UdpSocket::bind_udp((Ipv4Addr::UNSPECIFIED, 0).into(), opts)?;
// 멀티캐스트 전송 실패 처리
socket.send_to(message.as_bytes(), SSDP_MULTICAST_IP).await
.with_context(|| format!("failed to send SSDP search request to {SSDP_MULTICAST_


// 응답 파싱 실패 시 경고 로그
match parse_upnp_discover_response(response, addr) {
```

```
Ok(r) => tx.send(r)?,
Err(e) => warn!(response=?BStr::new(response), "failed to parse SSDP response: {
}
}
```

2. 포트 매핑 오류 처리

```
if let Err(e) = forward_port(control_url.clone(), local_ip, port, lease_duration
warn!("failed to forward port: {e:#}");
}
```

복구 전략:

- **재시도 메커니즘**: 실패 시 다음 주기에 자동 재시도
- **다중 엔드포인트**: 여러 UPnP 디바이스 동시 관리
- **로깅**: 상세한 오류 로그로 디버깅 지원

3. 네트워크 변경 대응

- **주기적 검색**: 60초마다 새로운 디바이스 검색
- **동적 IP 처리**: 네트워크 인터페이스 변경 시 자동 감지
- **중복 방지**: 이미 관리 중인 서비스 중복 실행 방지

성능 최적화

1. 비동기 처리

- **tokio::select!**: 여러 비동기 작업 동시 처리
- **FuturesUnordered**: 동적 태스크 관리
- **채널 통신**: 검색과 관리 작업 분리

2. 리소스 효율성

- **HashSet 중복 방지**: 동일 서비스 중복 관리 방지
- **주기적 갱신**: 리스 기간의 절반마다만 갱신
- **지연 처리**: MissedTickBehavior::Delay로 부하 분산

3. 네트워크 최적화

- **타임아웃 설정**: 10초 검색 타임아웃으로 응답성 보장
- **로컬 바인딩**: 특정 네트워크 인터페이스 바인딩 지원
- **IPv4 우선**: IPv6 지원 부족으로 IPv4 우선 사용

보안 고려사항

1. 네트워크 보안

- **로컬 네트워크 제한**: 멀티캐스트는 로컬 네트워크에만 전송
- **인증 없음**: UPnP는 기본적으로 인증 메커니즘 없음
- **포트 설명**: "rust UPnP"로 매핑 출처 식별 가능

2. 리소스 보호

- **포트 범위 제한**: 지정된 포트만 포워딩
- **리스 기간**: 60초 리스로 장기간 점유 방지
- **자동 해제**: 프로세스 종료 시 자동 해제 (리스 만료)

한계 및 개선 방안

1. 현재 한계

- **IPv6 미지원**: IPv4만 지원, IPv6 테스트 환경 부족
- **단일 프로토콜**: TCP만 지원, UDP 포트 포워딩 미지원
- **오류 복구**: 실패 시 단순 재시도만 수행

2. 개선 가능 영역

- **IPv6 지원**: 듀얼스택 환경에서 IPv6 UPnP 지원
- **UDP 지원**: DHT 등을 위한 UDP 포트 포워딩
- **백오프 전략**: 실패 시 지수 백오프 적용
- **상태 모니터링**: 포트 매핑 상태 확인 기능

결론

rqbit의 UPnP 포트 포워딩 구현은 표준 UPnP 프로토콜을 충실히 따르면서도 실용적인 최적화를 적용한 견고한 구현입니다. SSDP 검색부터 SOAP 기반 포트 매핑까지 전체 과정을 비동기적으로 처리하며, 네트워크 변경과 오류 상황에 대한 복구 메커니즘을 제공합니다. 특히 주기적 갱신과 다중 엔드포인트 지원을 통해 안정적인 포트 포워딩을 보장합니다.